JAVA RADIOBUTTONDEMO

RBD.java

Source History

```
1  package RBD;
2
3  import javax.swing.*;
4  import java.awt.*;
5
6  public class RBD extends JFrame {
7      String[] pets = {"Bird", "Cat", "Dog", "Rabbit", "Pig"};
8      JLabel img = new JLabel();
9
10     public static void main(String[] args) {
11         SwingUtilities.invokeLater(() -> new RBD().setVisible(true));
12     }
13
```

import java.swing.* --used for making windows buttons and popup message boxes

import java.awt.* -- used for layouts.

The “extends JFrame” shows that the created class would be in form of a window.

JLabel img = new Jlabel creates the space where the images would fit

```
9
10     public static void main(String[] args) {
11         SwingUtilities.invokeLater(() -> new RBD().setVisible(true));
12     }
13
```

this ensures that java waits for the window to be fully ready and make it visible.

```
    RBD() {
        setTitle("RadioButtonDemo");
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLayout(new BorderLayout());

        JPanel p = new JPanel(new GridLayout(5,1));
        ButtonGroup g = new ButtonGroup();
        JRadioButton last = null;

        for (String s : pets) {
            JRadioButton rb = new JRadioButton(s);
            g.add(rb);
            rb.addActionListener(e -> {
                img.setIcon(getImg(s));
                JOptionPane.showMessageDialog(this, "You selected: " + s);
            });
            p.add(rb);
            last = rb;
        }

        add(p, BorderLayout.WEST);
        add(img, BorderLayout.CENTER);
        setSize(350, 250);
```

The setTitle function puts the title at the top of the window

SetDefaultCloseOperation is used to fully close the window when the close button is pressed.

JPanel p = new Jpanel….Creates a container that holds the radio buttons.

ButtonGroup g …. ensures only one radio button can be selected at a time.

JRadioButton rb = new…. Creates a brand new radio button for the current pet name

g.add(rb);: Adds it to the ButtonGroup so it behaves like a regular radio button

rb.addActionListener(e -> { ... })...is the Event Listener. It means, *When someone clicks a particular button do the following two things..*

img.setIcon(getImg(s))... Update the right-side label with the picture for this pet.

JoptionPane.showMessageDialog….Pop up the required message box showing what animal was chosen.

p.add(rb)..Adds the button to the panel

last = rb.. Since it’s a loop, this line runs 5 times.

add(p, BorderLayout.WEST);: Puts the panel of buttons on the left side of the window.

add(img, BorderLayout.CENTER);: Puts the image label in the center

setSize(450, 250);: Tells the window to be 450 pixels wide and 250 pixels tall.

```
    ImageIcon getImg(String name) {
        try {
            String path = "/RBD/images/" + name.toLowerCase() + ".jpeg";
            java.net.URL url = getClass().getResource(path);
            if (url != null)
                return new ImageIcon(new ImageIcon(url).getImage().getScaledInstance(200, 200, Image.SCALE_SMOOTH));
        } catch (Exception e) {}
        return null;
    }
}
```

getImg(String name):….A helper function called whenever a button is clicked. It takes the pet name and goes to find the picture.

•try { ... } catch (Exception e) {}: This command tries *to find the image and If it fails it just ignores the error and moves on instead of crashing the whole app.*

•String path = "/RBD/images/" + name.toLowerCase() + ".jpeg"…. This is the exact file path..

•java.net.URL url = getClass().getResource(path);: Java looks into the NetBeans build folder to find this specific file path.

•getScaledInstance(150, 150, Image.SCALE_SMOOTH): A built-in Java command that resizes the image.

•return null;: If Java cannot find the image file, it returns nothing, leaving the right side blank